

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Repaso

Ajuste lineal con `np.polyfit`

Ejemplo: Ley de Hubble

Ajuste lineal con `curve_fit`

Relación masa–luminosidad

Interpolación

Ajuste de distribuciones

Ejercicios Guiados

Cierre

Introducción y Repaso

- **Tema central:** Ajuste de datos y modelado con NumPy y SciPy.
- **Objetivo:** Construir, paso a paso, una base sólida para ajustar datos experimentales y astronómicos.
- **Hoy (P1):**
 - Ajuste lineal con `np.polyfit` y `scipy.optimize.curve_fit`.
 - Ejemplos: Ley de Hubble, relación masa–luminosidad (transformación log–log).
 - Interpolación con `scipy.interpolate`.
 - Residuos y bondad de ajuste (χ^2 reducido).
- **Prerequisitos suaves:** `numpy` (arrays, operaciones vectorizadas) y `matplotlib`.

Introducción y Repaso $\ni$ Mini-repaso: arrays, ruido y gráficos

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Semilla reproducible
5 rng = np.random.default_rng(42)
6
7 # Datos sintéticos simples
8 x = np.linspace(0, 10, 25)
9 y_true = 2.5 * x + 5.0
10 sigma = 2.0 # incertidumbre típica de medición
11 y_obs = y_true + rng.normal(0, sigma, size=x.size)
12
13 plt.scatter(x, y_obs, s=20)
14 plt.xlabel('x'); plt.ylabel('y'); plt.grid(alpha=0.3)
15 plt.title('Datos con ruido')
16 plt.show()
```

Idea: simularemos escenarios físicos con ruido para practicar ajustes.

Ajuste lineal con `np.polyfit`

Ajuste lineal con `np.polyfit` $\ni$ Fenómeno físico: relación lineal simple

- **Escenarios típicos:** calibración sensor $V = aT + b$, ley de Hooke en pequeñas deformaciones $F = kx$, ley de Ohm $V = RI$.
- **¿Por qué una recta?** por *aproximación lineal* (expansión de Taylor de primer orden) en un rango acotado.
- **Qué medimos:** pares (x_i, y_i) con incertidumbre en y y (a veces) en x .
- **Meta:** estimar parámetros físicos (pendiente e intercepto) y evaluar calidad del modelo.

Ajuste lineal con `np.polyfit` $\ni$ Recta por mínimos cuadrados con `np.polyfit`

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Supongamos  $y = m x + b + \text{ruido}$ 
5 m_est, b_est = np.polyfit(x, y_obs, deg=1) # deg (int) grado del polinomio a ajustar
6 y_fit = m_est * x + b_est
7
8 res = y_obs - y_fit
9 SS_res = np.sum(res**2)
10 SS_tot = np.sum((y_obs - np.mean(y_obs))**2)
11 R2 = 1 - SS_res/SS_tot
12
13 plt.scatter(x, y_obs, label='datos')
14 plt.plot(x, y_fit, 'r', label=f'fit:  $y={m\_est:.2f}x+{b\_est:.2f}$ ')
15 plt.legend(); plt.grid(alpha=0.3); plt.show()
16 print('R^2 =', R2)
```

Notas: `np.polyfit` entrega coeficientes polinomiales por mínimos cuadrados.

Ajuste lineal con `np.polyfit` ∋ ¿Qué hace este código? (paso a paso)

- Genera datos x y una relación lineal $y_{\text{true}} = mx + b$ con ruido gaussiano (σ).
- Aplica `np.polyfit` para obtener m, b que minimizan $\sum r_i^2$.
- Calcula R^2 para medir la fracción de varianza explicada.
- Grafica datos y la recta ajustada para inspección visual.

Por qué así: es la forma estándar y eficiente de mínimos cuadrados en NumPy.

Ajuste lineal con `np.polyfit` $\ni$ ¿Qué hace realmente `polyfit`? y supuestos

- **Modelo:** asumimos una relación polinómica (aquí, recta)
 $y_i \approx mx_i + b$.
- **Objetivo:** encontrar m, b que *minimicen la suma de cuadrados* de los residuos: $\sum (y_i - y_{\text{mod}})^2$.
- **Supuestos implícitos:**
 - Errores en y son gaussianos, independientes y con varianza constante.
 - El modelo lineal es adecuado en el rango de datos.
- **Física:** estos supuestos se aproximan cuando la incertidumbre experimental es similar para todos los puntos y el fenómeno es lineal en el rango.
- **Resultado:** `polyfit` devuelve los coeficientes del polinomio que minimiza el error cuadrático medio.

Ajuste lineal con `np.polyfit` $\ni$ Ponderación por incertidumbres: $w = 1/\sigma$

```
1 # Si cada punto tiene una incertidumbre distinta sigma_i en y,  
2 # podemos ponderar el ajuste: puntos con menor sigma pesan más.  
3 rng = np.random.default_rng(0)  
4 sigma_i = rng.uniform(1.0, 3.0, size=x.size)  
5 w = 1.0 / sigma_i  
6  
7 m_w, b_w = np.polyfit(x, y_obs, deg=1, w=w)  
8 print(f"sin peso: m={m_est:.2f}, b={b_est:.2f}")  
9 print(f"con peso: m={m_w:.2f}, b={b_w:.2f}")
```

Interpretación: en física, mediciones más precisas deben influir más en el ajuste.

Ejemplo: Ley de Hubble

Ejemplo: Ley de Hubble $\ni$ Ley de Hubble: contexto físico y observacional

- **Idea simple:** cuanto más **lejos** está una galaxia (d), más **rápido** se aleja (v). La relación es $v = H_0 d$.
- **Interpretación geométrica:** el *pendiente* de la recta es H_0 . Si la expansión fuera perfecta, el *intercepto* sería 0.
- **Régimen de validez local:** para $z \ll 1$, $v \approx cz$. Para z grandes se usa un modelo cosmológico completo.
- **Cómo se mide:**
 - v : a partir del **corrimiento al rojo** z medido en espectros.
 - d : con la **escalera de distancias** (cefeidas, supernovas Ia, etc.).
- **Dispersión real:** las **velocidades peculiares** ($\sim 100\text{--}300$ km/s) añaden ruido, especialmente a distancias pequeñas.

Diccionario rápido (símbolos y unidades)

- v : velocidad de recesión, en km/s.
- d : distancia, en Mpc ($1 \text{ Mpc} \approx 3.26$ millones de años luz).
- H_0 : constante de Hubble, en $\text{km s}^{-1} \text{Mpc}^{-1}$ (~ 70 en este ejemplo).
- z : corrimiento al rojo; si $z \ll 1$, $v \approx cz$ (c : velocidad de la luz).
- Intercepto: valor de v cuando $d = 0$; idealmente ≈ 0 , pero puede verse afectado por sesgos/peculiares.

Ejemplo numérico

Si $H_0 = 70 \text{ km s}^{-1} \text{Mpc}^{-1}$ y $d = 100 \text{ Mpc}$, entonces $v \approx 7000 \text{ km/s}$.

Ejemplo: Ley de Hubble $\ni$ Supuestos, validez y fuentes de dispersión

- **Supuestos:** el universo es *más o menos igual* en todas partes (homogéneo e isótropo) y se expande de forma uniforme *en promedio*.
- **Unidades y tiempos:** H_0 también puede escribirse en s^{-1} . El *tiempo de Hubble* $t_H \approx 1/H_0$ da una escala de edad del universo.
- En el ejemplo siguiente usaremos datos **sintéticos** para practicar el ajuste y recuperar H_0 .

Traducción simple

"Más lejos $\Rightarrow$ más rápido". El H_0 es cuántos km/s aumentan por cada Mpc de distancia.

Cuidado

A distancias pequeñas los términos peculiares pueden dominar y hacer que el intercepto aparente no sea 0.

Ejemplo: Ley de Hubble $\ni$ Ley de Hubble: $v = H_0 d$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 # Simular galaxias: velocidad vs distancia
4 rng = np.random.default_rng(7)
5 H0_true = 70.0 # km/s/Mpc
6 d = np.linspace(0, 200, 30) # Mpc
7 v_true = H0_true * d
8 sigma_v = 150.0 # km/s (dispersión peculiar)
9 v_obs = v_true + rng.normal(0, sigma_v, size=d.size)
10
11 # Graficar datos observados
12 plt.scatter(d, v_obs, s=20)
13 plt.xlabel('Distancia (Mpc)'); plt.ylabel('Velocidad (km/s)')
14
15 # Ajustar H0 a los datos simulados
16 H0_hat, v0_hat = np.polyfit(d, v_obs, 1)
17 v_fit = H0_hat * d + v0_hat
18 plt.plot(d, v_fit, 'r', label=f'fit: H0={H0_hat:.1f} km/s/Mpc')
19
20 print(f'H0 estimado = {H0_hat:.1f} km/s/Mpc, intercepto = {v0_hat:.1f} km/s")
```

Física: a primer orden, la expansión se ve como una relación lineal entre v y d .

Ejemplo: Ley de Hubble $\ni$ ¿Qué hace este código de Hubble?

- Simula distancias d y velocidades v con H_0 verdadero, agregando **dispersión peculiar** (ruido) σ_v .
- Ajusta $v = H_0 d + b$ para estimar H_0 e intercepto.
- Muestra la recta resultante y reporta los parámetros.
- Posteriormente calcula residuos y χ^2_{red} para evaluar consistencia con las σ_v .

Conexión física: el pendiente es una medida de la expansión local del Universo.

Ejemplo: Ley de Hubble $\ni$ Ley de Hubble: residuos y χ^2 reducido

```
1 # Residuos y  $\chi^2$  reducido del ajuste lineal  $v = H_0 d + b$ 
2 res = v_obs - v_fit
3 chi2 = np.sum((res/sigma_v)**2)
4 nu = d.size - 2 # parámetros:  $H_0$  y  $b$ 
5 chi2_red = chi2/nu
6 print(f"chi2_red = {chi2_red:.2f}")
7
8 # (Opcional) Inspección de residuos por distancia
9 # for di, ri in zip(d, res):
10 #     print(f"d={di:6.1f} Mpc, residuo={ri:7.1f} km/s")
```

Lectura física: residuos reflejan velocidades peculiares y errores en distancias; $\chi_{\text{red}}^2 \approx 1$ sugiere incertidumbres realistas.

Ejemplo: Ley de Hubble $\ni$ Residuos en contexto físico

- **Residuos** $r_i = v_{\text{obs},i} - v_{\text{fit},i}$: nos dicen cuánto se desvía cada galaxia de la recta de Hubble.
- **Causas típicas**: velocidades peculiares (caídas al pozo de un cúmulo, interacciones), errores en distancias, selección de muestra.
- **Patrones**: si los residuos muestran tendencia con d , el modelo lineal podría ser insuficiente o las incertidumbres están mal estimadas.
- χ^2 **reducido cercano a 1**: sugiere que las σ usadas reflejan bien la dispersión real.

Ajuste lineal con `curve_fit`

Ajuste lineal con `curve_fit` $\ni$ `scipy.optimize.curve_fit` en un modelo lineal

```
1 from scipy.optimize import curve_fit
2
3 def modelo_lineal(x, m, b):
4     return m*x + b
5
6 popt, pcov = curve_fit(modelo_lineal, d, v_obs, sigma=sigma_v,
7                        absolute_sigma=True)
8 m_cf, b_cf = popt
9
10 # Errores estándar de los parámetros
11 # np.diag(pcov) extrae la varianza de cada parámetro
12 perr = np.sqrt(np.diag(pcov))
13
14 # Graficar resultados del ajuste
15 plt.scatter(d, v_obs, s=20, label='galaxias')
16 plt.plot(d, modelo_lineal(d, *popt), 'r', label=f'fit: m={m_cf:.2f}, b={b_cf:.1f}')
17
18 print(f"m={m_cf:.2f}±{perr[0]:.2f}, b={b_cf:.1f}±{perr[1]:.1f}")
```

Ventaja: entrega covarianzas de parámetros para estimar errores.

Relación masa–luminosidad

Relación masa–luminosidad $\ni$ Potencia: $L = k M^\alpha$. Enfoque log–log

```
1 # Simular estrellas
2 rng = np.random.default_rng(1)
3 alpha_true = 3.5
4 k_true = 1.0
5 M = np.linspace(0.5, 10, 40)
6 L_true = k_true * M**alpha_true
7 sigma_rel = 0.10
8 L_obs = L_true * (1 + rng.normal(0, sigma_rel,
  ↳ size=M.size))
9
10 # Transformación log10
11 X = np.log10(M)
12 Y = np.log10(L_obs)
13 a, b = np.polyfit(X, Y, 1) # Y = a X + b
```

```
16 alpha_hat = a
17 k_hat = 10**b
18 print(f"alpha≈{alpha_hat:.2f}, k≈{k_hat:.2f}")
19
20 import matplotlib.pyplot as plt
21 plt.scatter(X, Y, s=20)
22 plt.plot(X, a*X + b, 'r')
23 plt.xlabel('log M'); plt.ylabel('log L')
24 plt.title('Masa–Luminosidad (log–log)')
25 plt.grid(alpha=0.3); plt.show()
```

Discusión: la linealización facilita el ajuste de leyes de potencia.

Relación masa–luminosidad $\ni$ ¿Qué hace este código (log–log)?

- Simula M y L con una ley de potencia y ruido multiplicativo (más realista en astrofísica).
- Transforma a $\log$ para linealizar: $\log L = \alpha \log M + \log k$.
- Ajusta con **polyfit** en el espacio logarítmico y recupera α y k .
- Discute diferencias con el ajuste no lineal en el espacio original.

Relación masa–luminosidad $\ni$ Potencia directa con `curve_fit` (no lineal)

```
1 from scipy.optimize import curve_fit
2
3 def power_law(M, k, alpha):
4     return k * M**alpha
5
6 popt, pcov = curve_fit(power_law, M, L_obs, p0=(1.0, 3.0))
7 k_cf, alpha_cf = popt
8 perr = np.sqrt(np.diag(pcov))
9 print(f"k={k_cf:.2f}±{perr[0]:.2f}, alpha={alpha_cf:.2f}±{perr[1]:.2f}")
10 # Graficar resultados
11 plt.scatter(M, L_obs, s=20, label='estrellas')
12 M_fit = np.linspace(0.5, 10, 100)
13 plt.plot(M_fit, power_law(M_fit, *popt), 'r', label='fit potencia')
14
```

Comparación: log–log vs ajuste no lineal; considerar sesgos con ruido multiplicativo.

- α captura cómo cambia la **producción de energía** con la masa (tasa de fusión y estructura interna).
- Para estrellas de la secuencia principal, típicamente $\alpha \approx 3\text{--}4$ en ciertos rangos de masa.
- **Escalas y unidades:** cuidado con trabajar en unidades consistentes; la forma funcional es adimensional si se normaliza.
- **Log–log vs no lineal:** la transformación logarítmica linealiza pero cambia la distribución de errores; el ajuste no lineal trabaja en el espacio original.

Interpolación

Interpolación $\ni$ Interpolación 1D con `interp1d`

```
1 import numpy as np
2 from scipy.interpolate import interp1d
3 import matplotlib.pyplot as plt
4
5 x = np.linspace(0, 10, 12)
6 y = np.sin(x) + 0.1*np.cos(3*x)
7 f_lin = interp1d(x, y, kind='linear')
8 f_cub = interp1d(x, y, kind='cubic')
9
10 xx = np.linspace(0, 10, 200)
11 plt.plot(x, y, 'o', label='muestras')
12 plt.plot(xx, f_lin(xx), '-', label='lineal')
13 plt.plot(xx, f_cub(xx), '--', label='cúbica')
14 plt.legend(); plt.grid(alpha=0.3); plt.show()
```

Uso típico: rellenar curvas (p. ej., espectros o curvas de luz) entre puntos.

Interpolación $\ni$ ¿Qué hace este código de interpolación?

- Construye una señal suave $y(x)$ muestreada en pocos puntos.
- Define interpoladores lineal y cúbico, y evalúa en una malla densa.
- Compara ambas curvas para decidir según la física del fenómeno (suavidad esperable).

Interpolación $\ni$ Interpolación: polinomio de alto grado (mal) vs cúbica (bien)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Función tipo Runge con ruido
5 rng = np.random.default_rng(0)
6 x = np.linspace(-1, 1, 11)
7 y = 1/(1 + 25*x**2) + rng.normal(0, 0.02, x.size)
8
9 # MAL: polinomio de alto grado
10 deg = 10
11 p = np.polyfit(x, y, deg)
12 xx = np.linspace(-1, 1, 400)
13 yy_poly = np.polyval(p, xx)
14
15 plt.plot(x, y, 'o', label='muestras')
16 plt.plot(xx, yy_poly, 'r-', label=f'pol grado
   ↪ {deg}')
17 plt.title('Oscilaciones (Runge)')
18 plt.legend(); plt.grid(True, alpha=0.3); plt.show()
```

```
1 # BIEN: interpolación cúbica por tramos
2 from scipy.interpolate import interp1d
3 f_cub = interp1d(x, y, kind='cubic') # corrección
   ↪ clave
4
5 plt.plot(x, y, 'o', label='muestras')
6 plt.plot(xx, f_cub(xx), 'g-', label='interp
   ↪ cúbica')
7 plt.title('Cúbica estable (sin oscilaciones)')
8 plt.legend(); plt.grid(True, alpha=0.3); plt.show()
```

Idea: polinomios de grado alto suelen oscilar; preferir esquemas por tramos (cúbica, splines).

Interpolación $\ni$ ¿Qué error vemos? Fenómeno de Runge

- **Síntoma:** la curva ajustada con un polinomio global de alto grado **oscila** mucho, sobre todo en los extremos, pese a pasar cerca de los puntos.
- **Causa:** polinomios globales son rígidos y sensibles a nodos equiespaciados; pequeñas variaciones en datos generan grandes ondulaciones.
- **Consecuencia:** la forma física se distorsiona; el error entre nodos puede ser grande aunque el error en nodos sea pequeño.
- **Buenas prácticas:**
 - Usar **interpolación por tramos** (cúbica, splines B) o **PCHIP** si se requiere preservar forma.
 - Si se exige polinomio global, preferir **nodos de Chebyshev** en lugar de equiespaciados.
 - Validar visualmente y con métricas locales, no solo con R^2 .

Interpolación $\ni$ Extrapolación peligrosa vs límites seguros

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.interpolate import interp1d
4
5 x = np.linspace(0, 10, 8)
6 y = np.exp(-0.3*x)
7 f_lin = interp1d(x, y, kind='linear',
8                 fill_value='extrapolate')
9 xx = np.linspace(-2, 12, 300)
10
11 plt.plot(x, y, 'o', label='muestras')
12 plt.plot(xx, f_lin(xx), 'r-', label='extrapola')
13 plt.axvspan(-2, 0, color='red', alpha=0.05)
14 plt.axvspan(10, 12, color='red', alpha=0.05)
15 plt.title('Fuera de rango: engañoso')
16 plt.legend(); plt.grid(True, alpha=0.3); plt.show()
```

```
1 # Corrección: no extrapolar
2 f_safe = interp1d(x, y, kind='linear',
3                  bounds_error=False,
4                  fill_value=np.nan) # clave
5
6 plt.plot(x, y, 'o')
7 plt.plot(xx, f_safe(xx), 'g-', label='sin
  ↪ extrapolar')
8 plt.title('Restringir al rango de datos')
9 plt.legend(); plt.grid(True, alpha=0.3); plt.show()
```

Regla: evitar extrapolar a menos que el modelo físico lo justifique.

Interpolación $\ni$ ¿Por qué extrapolar es peligroso?

- **Fuera del rango no hay datos:** la pendiente y curvatura son *suposiciones*, no información.
- **Amplificación de error:** pequeños errores en el ajuste dentro del rango pueden crecer mucho fuera de él.
- **Modelos físicos primero:** solo extrapole si un **modelo físico** bien justificado describe el comportamiento.
- **Prácticas seguras:**
 - Use `bounds_error=False`, `fill_value=np.nan` para no “inventar” valores.
 - Señalice visualmente regiones extrapoladas si las muestra.
 - Si debe extrapolar, **propague incertidumbre** y reporte intervalos amplios.

Interpolación $\ni$ Conservar forma física: cúbica vs PCHIP (monótono)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.interpolate import interp1d
4
5 x = np.linspace(0, 6, 12)
6 y = 1 - np.exp(-x) + 0.02*np.sin(6*x)
7 f_cub = interp1d(x, y, kind='cubic')
8 xx = np.linspace(0, 6, 400)
9
10 plt.plot(x, y, 'o')
11 plt.plot(xx, f_cub(xx), 'r-', label='cúbica')
12 plt.title('Overshoot (viola monotonicidad)')
13 plt.legend(); plt.grid(True, alpha=0.3); plt.show()
```

```
1 # Corrección: PCHIP preserva forma/monot.
2 from scipy.interpolate import PchipInterpolator
3 f_mon = PchipInterpolator(x, y) # clave
4
5 plt.plot(x, y, 'o')
6 plt.plot(xx, f_mon(xx), 'g-', label='PCHIP')
7 plt.title('PCHIP: sin overshoot')
8 plt.legend(); plt.grid(True, alpha=0.3); plt.show()
```

Uso: cuando el fenómeno debe ser monótono o libre de oscilaciones no físicas.

Interpolación $\ni$ ¿Qué es overshoot? ¿Por qué PCHIP ayuda?

- **Overshoot/undershoot:** la interpolación **sobrepasa** (o cae por debajo de) el rango definido por puntos vecinos, creando crestas/valles *no físicos*.
- **Cúbica estándar:** prioriza suavidad de derivadas pero puede introducir oscilaciones y overshoot cerca de cambios de curvatura.
- **PCHIP:** interpolador por tramos que **preserva la forma y la monotonidad**; evita overshoot manteniendo la curva dentro del rango local.
- **Cuándo usar:** series que deben ser **monótonas** o sin oscilaciones artificiales (curvas de respuesta, acumulados, luminosidad vs tiempo sin rebotes).
- **Trade-off:** PCHIP puede ser menos suave en derivadas que un spline cúbico, pero más fiel físicamente.

Ajuste de distribuciones

Ajuste de distribuciones $\ni$ ¿Por qué ajustar distribuciones? (física/astro)

- **Ruido térmico** y errores de medición: a menudo **normal** (gaussiano).
- **Flujos/energías multiplicativos** y **masas** positivas: suelen ser **log-normales**.
- **Tiempos de espera** para eventos aleatorios: **exponencial**.
- **Conteos de fotones**: Poisson (discreta, fuera del alcance de este ejemplo continuo).
- Elegir bien la distribución **mejora la modelación de incertidumbre** y la **inferencia** posterior.

Ajuste de distribuciones $\ni$ Generar datos sintéticos: normal, log-normal, exponencial

```
1 import numpy as np
2 rng = np.random.default_rng(42)
3
4 # Normal (media=0, sigma=1)
5 x_norm = rng.normal(0, 1, size=2000)
6
7 # Lognormal: si  $Z \sim N(\mu, s)$ ,  $X = \exp(Z) \Rightarrow \log X \sim N(\mu, s)$ 
8 mu, s = 0.0, 0.6
9 x_logn = rng.lognormal(mean=mu, sigma=s, size=2000)
10
11 # Exponencial (escala=tau)
12 tau = 2.0
13 x_exp = rng.exponential(scale=tau, size=2000)
```

Nota: elegir tamaños suficientes para estimación estable de parámetros.

Ajuste de distribuciones $\ni$ Ajustar parámetros con `scipy.stats`

```
1 from scipy import stats
2
3 # Normal: params = (mu_hat, sigma_hat)
4 mu_hat, sigma_hat = stats.norm.fit(x_norm)
5
6 # Lognormal en SciPy: shape=s, loc, scale;  $X \sim \text{lognorm}(s, \text{loc}, \text{scale})$ 
7 s_hat, loc_hat, scale_hat = stats.lognorm.fit(x_logn, floc=0)
8 mu_hat_logn = np.log(scale_hat) # equivalente a mu del log
9
10 # Exponencial: params = (loc, scale=tau)
11 loc_e, tau_hat = stats.expon.fit(x_exp, floc=0)
12
13 print(mu_hat, sigma_hat)
14 print(s_hat, mu_hat_logn)
15 print(tau_hat)
```

Por qué: `fit` maximiza verosimilitud para obtener estimadores eficientes bajo supuestos.

Ajuste de distribuciones $\ni$ Visualizar y comparar: histograma + PDFs (datos normales)

```
1 import numpy as np, matplotlib.pyplot as plt
2 from scipy import stats
3
4 # Datos normales simulados
5 rng = np.random.default_rng(42)
6 x = rng.normal(0, 1, 2000)
7
8 # Ajustes candidatos
9 mu, sig = stats.norm.fit(x)
10 s, _, scale = stats.lognorm.fit(x)
11 _, tau = stats.expon.fit(x)
12
13 # Curvas de densidad
14 xs = np.linspace(x.min(), x.max(), 400)
15 pdf_norm = stats.norm.pdf(xs, mu, sig)
16 pdf_logn = stats.lognorm.pdf(xs, s, 0, scale)
17 pdf_exp = stats.expon.pdf(xs, 0, tau)
18
19 # Gráfico comparativo
20 plt.hist(x, bins='auto', density=True, alpha=0.3, label='hist')
21 plt.plot(xs, pdf_norm, 'r', label='normal')
22 plt.plot(xs, pdf_logn, 'g--', label='lognormal')
23 plt.plot(xs, pdf_exp, 'k:', label='exponencial')
24 plt.legend(); plt.grid(True, alpha=0.3); plt.show()
```

Regla: siempre graficar el histograma y superponer PDFs ajustadas.

Ajuste de distribuciones $\ni$ Visualizar y comparar: histograma + PDFs (datos lognormales)

```
1 import numpy as np, matplotlib.pyplot as plt
2 from scipy import stats
3
4 # Datos lognormales simulados
5 rng = np.random.default_rng(0)
6 x = rng.lognormal(mean=0.0, sigma=0.6, size=2000)
7
8 # Ajustes candidatos
9 s, _, scale = stats.lognorm.fit(x, floc=0)
10 mu_n, sig_n = stats.norm.fit(x)
11 _, tau = stats.expon.fit(x, floc=0)
12
13 xs = np.linspace(0, np.percentile(x, 99.5), 400)
14 pdf_logn = stats.lognorm.pdf(xs, s, 0, scale)
15 pdf_norm = stats.norm.pdf(xs, mu_n, sig_n)
16 pdf_exp = stats.expon.pdf(xs, 0, tau)
17
18 plt.hist(x, bins='auto', density=True, alpha=0.3, label='hist')
19 plt.plot(xs, pdf_logn, 'g', label='lognormal')
20 plt.plot(xs, pdf_norm, 'r--', label='normal')
21 plt.plot(xs, pdf_exp, 'k:', label='exponencial')
22 plt.legend(); plt.grid(True, alpha=0.3); plt.show()
```

Lectura: compare forma de colas y sesgo; el mejor ajuste debe seguir la masa del histograma.

- **Primero:** histograma con densidad (`density=True`) y PDFs ajustadas superpuestas.
- **Luego:** CDF empírica (ECDF) vs CDF del modelo (como en KS) para ver desviaciones en colas.
- **Opcional:** gráficos Q-Q/P-P para inspección fina de colas.
- **Comparar varios candidatos** (normal/lognormal/exponencial) en el **mismo eje** antes de decidir.
- Acompañar con un criterio cuantitativo (AIC/BIC/KS) pero **no decidir sin visualización**.

Ajuste de distribuciones $\ni$ Bondad de ajuste: KS y verosimilitud (AIC)

```
1 from scipy import stats
2 import numpy as np
3
4 # KS para normal
5 D, p = stats.kstest(x_norm, 'norm', args=(mu_hat, sigma_hat))
6
7 # AIC:  $2k - 2 \ln L$ 
8 ll_norm = np.sum(stats.norm.logpdf(x_norm, mu_hat, sigma_hat))
9 k = 2
10 AIC_norm = 2*k - 2*ll_norm
11 print(D, p, AIC_norm)
```

Lectura: p grande no rechaza; AIC menor suele preferirse entre modelos comparables.

Ajuste de distribuciones $\ni$ ¿Qué hace el código y por qué?

- **Generación:** construimos datos para escenarios típicos (normal, lognormal, exponencial).
- **Ajuste:** usamos `.fit` de SciPy (MLE) para recuperar parámetros.
- **Diagnóstico:** KS verifica distancia entre CDFs; AIC compara modelos por parsimonia.
- **Cuidado:** lognormal en SciPy usa parámetros *shape s*, *loc*, *scale* (no directamente μ, σ).

Ajuste de distribuciones $\ni$ Notas prácticas de interpolación en física/astro

- **No extrapolar:** fuera del rango de datos la interpolación se vuelve especulación.
- **Curvas de luz:** útil para rellenar huecos antes de integrar o ajustar un período.
- **Espectros:** alinear rejillas (grid) antes de comparar o integrar flujo en bandas.
- **Suavidad vs fidelidad:** `cubic` suaviza más que `linear`; elegir según física del fenómeno.

Ejercicios Guiados

Ejercicios Guiados $\ni$ Ejercicio 1:

Calibración de sensor lineal



Enunciado

- Un sensor entrega voltaje V proporcional a temperatura T :
 $V = aT + b$.
- Con 10 pares (T_i, V_i) y $\sigma_V = 0.02\text{ V}$, estime a, b con `np.polyfit` y calcule R^2 y χ^2_{red} .
- Interprete a y b físicamente (sensibilidad y offset).

Objetivo: práctica de ajuste lineal y métricas de calidad.

Ejercicios Guiados $\ni$ Solución 1 de Referencia:

Calibración de sensor



```
1 import numpy as np
2 rng = np.random.default_rng(0)
3 T = np.linspace(0, 100, 10)
4 a_true, b_true, sigma = 0.015, 0.20, 0.02
5 V = a_true*T + b_true + rng.normal(0, sigma, size=T.size)
6 m, b = np.polyfit(T, V, 1)
7 Vf = m*T + b
8 R2 = 1 - np.sum((V-Vf)**2)/np.sum((V-V.mean())**2)
9 chi2r = np.sum(((V-Vf)/sigma)**2)/(T.size-2)
10 print(m, b, R2, chi2r)
```

Discusión: m es sensibilidad $V/^{\circ}\text{C}$; b es voltaje a 0°C .

Ejercicios Guiados $\ni$ Ejercicio 2:

Estimar H_0 con ponderación



Enunciado

- Genere 30 galaxias con dispersión peculiar σ_v variable (100–300 km/s).
- Ajuste $v = H_0 d + b$ sin y con pesos $w = 1/\sigma_v$. Compare H_0 y χ^2_{red} .
- Comente por qué el ponderado es más realista.

Objetivo: introducir pesos según incertidumbre.

Ejercicios Guiados $\ni$ Solución 2 de Referencia:

Estimar H_0 con ponderación



```
1 import numpy as np
2 rng = np.random.default_rng(7)
3 H0_true = 70.0
4 d = np.linspace(0, 200, 30)
5 sigma_v = rng.uniform(100, 300, size=d.size)
6 v_obs = H0_true*d + rng.normal(0, sigma_v)
7
8 # Sin pesos
9 m0, b0 = np.polyfit(d, v_obs, 1)
10 res0 = v_obs - (m0*d + b0)
11 chi2r0 = np.sum((res0/sigma_v)**2)/(d.size-2)
12
13 # Con pesos w = 1/sigma
14 w = 1.0/sigma_v
15 m1, b1 = np.polyfit(d, v_obs, 1, w=w)
16 res1 = v_obs - (m1*d + b1)
17 chi2r1 = np.sum((res1/sigma_v)**2)/(d.size-2)
18 print(m0, chi2r0, m1, chi2r1)
```

Comentario: el ajuste ponderado privilegia mediciones más precisas.

Ejercicios Guiados $\ni$ Ejercicio 3: Interpolación de curva de luz



Enunciado

- Simule una curva de luz $F(t)$ con huecos (muestras perdidas).
- Interpole (lineal y cúbica) y compare el flujo integrado (regla del trapecio) en un intervalo.
- Discuta cuál método conserva mejor el área y por qué.

Objetivo: conectar interpolación con magnitudes integradas.

Ejercicios Guiados $\ni$ Solución 3 de Referencia:

Interpolación y área



```
1 import numpy as np
2 from scipy.interpolate import interp1d
3
4 rng = np.random.default_rng(0)
5 t_full = np.linspace(0, 10, 201)
6 F_true = 1.0 + 0.1*np.sin(2*np.pi*t_full/2.5)
7
8 # Muestras con huecos
9 mask = rng.random(t_full.size) > 0.3
10 t = t_full[mask]; F = F_true[mask]
11
12 f_lin = interp1d(t, F, kind='linear', fill_value='extrapolate')
13 f_cub = interp1d(t, F, kind='cubic', fill_value='extrapolate')
14
15 A_true = np.trapz(F_true, t_full)
16 A_lin = np.trapz(f_lin(t_full), t_full)
17 A_cub = np.trapz(f_cub(t_full), t_full)
18 print(A_true, A_lin, A_cub)
```

Comparación: elegir esquema según suavidad real del fenómeno.



Enunciado

- Genere datos monótonos con leve ruido suave:
 $y = 1 - e^{-x} + 0.02 \sin(6x)$.
- Interpole con **cúbica** y con **PCHIP**. Verifique si cada interpolación es monótona.
- Discuta implicancias físicas del overshoot y cuándo preferir PCHIP.

Objetivo: preservar forma/monotonicidad en fenómenos físicos.

Ejercicios Guiados $\ni$ Solución 4 de Referencia:

Monotonidad



```
1 import numpy as np
2 from scipy.interpolate import interp1d, PchipInterpolator
3
4 x = np.linspace(0, 6, 12)
5 y = 1 - np.exp(-x) + 0.02*np.sin(6*x)
6 xx = np.linspace(0, 6, 400)
7
8 f_cub = interp1d(x, y, kind='cubic')
9 f_pch = PchipInterpolator(x, y)
10
11 is_mono_cub = np.all(np.diff(f_cub(xx)) >= -1e-8)
12 is_mono_pch = np.all(np.diff(f_pch(xx)) >= -1e-8)
13 print('cúbica monótona?', is_mono_cub)
14 print('PCHIP monótona?', is_mono_pch)
```

Resultado: PCHIP suele evitar overshoot y mantener monotonidad.

Ejercicios Guiados $\ni$ Ejercicio 5:

Ajuste normal: KS y AIC



Enunciado

- Genere 1500 datos de una normal desconocida (media y sigma).
- Estime μ, σ con `scipy.stats.norm.fit`.
- Calcule estadístico KS y el AIC del modelo normal.

Objetivo: evaluar bondad de ajuste con KS y AIC.

Ejercicios Guiados $\ni$ Solución 5 de Referencia:

Normal: KS y AIC



```
1 import numpy as np
2 from scipy import stats
3 rng = np.random.default_rng(42)
4 x = rng.normal(0.0, 1.2, 1500)
5
6 mu_hat, sig_hat = stats.norm.fit(x)
7 D, p = stats.kstest(x, 'norm', args=(mu_hat, sig_hat))
8 ll = np.sum(stats.norm.logpdf(x, mu_hat, sig_hat))
9 AIC = 2*2 - 2*ll # k=2 parámetros
10 print(f"mu={mu_hat:.2f}, sigma={sig_hat:.2f}")
11 print(f"KS: D={D:.3f}, p={p:.3f}")
12 print(f"AIC={AIC:.1f}")
```

Lectura: p grande no rechaza; AIC sirve para comparar con otros modelos.

Ejercicios Guiados $\ni$ Ejercicio 6:

Datos positivos: lognormal vs exponencial



Enunciado

- Genere 1500 datos positivos (tipo lognormal).
- Ajuste **lognormal** y **exponencial** (`scipy.stats`).
- Compare AIC y concluya qué modelo describe mejor los datos.

Objetivo: seleccionar modelo de distribución para datos positivos.

Ejercicios Guiados $\ni$ Solución 6 de Referencia:

Comparación AIC



```
1 import numpy as np
2 from scipy import stats
3 rng = np.random.default_rng(0)
4 x = rng.lognormal(mean=0.0, sigma=0.6, size=1500)
5
6 # Lognormal (fix loc=0)
7 s, _, scale = stats.lognorm.fit(x, floc=0)
8 ll_logn = np.sum(stats.lognorm.logpdf(x, s, 0, scale))
9 AIC_logn = 2*2 - 2*ll_logn # k=2: (s, scale)
10
11 # Exponencial (loc=0)
12 _, tau = stats.expon.fit(x, floc=0)
13 ll_exp = np.sum(stats.expon.logpdf(x, 0, tau))
14 AIC_exp = 2*1 - 2*ll_exp # k=1: (tau)
15
16 print(f"lognormal: s={s:.2f}, mu_log={np.log(scale):.2f}, AIC={AIC_logn:.1f}")
17 print(f"exponencial: tau={tau:.2f}, AIC={AIC_exp:.1f}")
```

Criterio: AIC menor indica el modelo preferido.

Cierre

- Vimos ajustes lineales con `np.polyfit` y `curve_fit`.
- Ejemplos físicos: Ley de Hubble y masa–luminosidad.
- Interpolación básica y análisis de residuos con χ^2 .
- Próxima sesión (P2): ajustes no lineales, propagación de errores e intervalos de confianza.

- NumPy polyfit
- SciPy curve_fit
- SciPy interpolate
- Matplotlib para visualización

¡Buen trabajo hasta la próxima sesión!

- Practicar con datasets propios (p. ej., espectros, curvas de luz, relaciones escalares).
- Traer dudas para profundizar en P2.